

Productive Failure Report

Hypothesis

A naive memory system (store everything, one similarity signal, no recency, no isolation, no admission gate, no decay, no abstention) will reproduce the D1 failures as measured numbers, and they will worsen as the store grows. The pre-registered plan is in `baseline_protocol.md`; I wrote the hypothesis before reading results.

Baseline design and workload

TF-IDF cosine over raw text, top-5, rebuilt at four store sizes (≈63, 207, 783, 1935 memories). A seeded filler haystack creates realistic pollution; on top sit labelled scenario memories and a fixed probe set with ground truth, covering all six required scenarios (pollution, stale preference, context budget, cross-tenant, PII, cold-start). Every probe has a known correct and, where relevant, a known poison answer, so failures are measured against ground truth rather than judged.

Results — the indictment

Headline failure rates at the largest store (1,935 memories):

Failure	Metric	Result	Reading
F2 — pollution	precision@5	0.20	The needle is found (hit@5 = 1.0, MRR = 0.94) but 80% of injected context is irrelevant filler . The failure is noise, not a miss.
F3 — stale preference	stale-wins rate	0.33	1 in 3 changed-preference probes ranks the outdated preference above the current one. With no recency signal it is a wording coin-flip.
F5 — cross-tenant leak	leak rate	0.92	11 of 12 personal queries return at least one other user's memory in the top-5. This is a security failure.
F6 — PII	stored / exposed	2/2 · 1.0	Both secrets (API key, card number) were stored with no filter and are retrievable .
Cold-start	abstention-failure	1.00	The system never abstains . It injects 5 memories even when nothing relevant exists.
F7 — growth	store / index	unbounded	Memories 63 → 1,935; index entries 248 → 6,836; text 2.5 → 70 KB. No decay, linear growth.

Two failures get worse as the store grows. Cross-tenant leakage rises from **0.25 at 63 memories to 0.92 at 1,935** — the more the system stores, the more often it leaks. Storage grows without bound. These are not constant annoyances; they are scaling liabilities.

Store size	63	207	783	1,935
Cross-tenant leak rate	0.25	0.25	0.92	0.92
Index entries (TF-IDF nnz)	248	743	2,745	6,836
Raw text (KB)	2.5	7.6	28.2	70.0
precision@5	0.20	0.20	0.20	0.20

Cold-start: the system guesses with a straight face. On unanswerable probes, mean top-1 similarity is **0.21**, versus **0.51** when an answer genuinely exists — less than half the confidence — yet the low-similarity memories are injected with identical authority. The zero-memory new user is a double failure: it returns *other* users' memories (no abstention *and* a leak).

What did NOT fail at this scale (honest caveats)

Two failures I predicted did not appear in the tested range, and saying so is part of the experiment.

Latency. p95 retrieval stayed ≈0.5–0.7 ms and did *not* grow with store size. At ≤2k memories a naive linear scan is sub-millisecond; the latency side of F7 is a *projection* beyond this range, not something I measured. It would bite at millions of memories, not thousands.

Context overflow. The relevant memory was never dropped under a 150-token budget (rate 0.0), because these memories are short (~11 tokens each). The real budget finding is different and more interesting: mean injected context is ≈53 tokens, of which ~80% is noise (from precision 0.20), and on unanswerable queries *100%* of the injected budget is wasted on guesses. The failure is **wasted budget, not overflowed budget** — which corrected my hypothesis.

Failure taxonomy

Observed failure	Violated assumption	User / system impact
80% of context is filler	Storing everything is fine; ranking will filter	Wasted tokens and cost; the model reasons over mostly-irrelevant context
Stale preference wins 1/3 of the time	Similarity = relevance; recency does not matter	Assistant acts on preferences the user explicitly changed; trust erodes
Leak rate reaches 0.92	Users are naturally separated	One user's data reaches another; catastrophic and irreversible

Secrets stored and retrievable	Nothing sensitive gets pasted	Retention and compliance liability that grows silently
Never abstains	Top-k is always good enough	Confident wrong context on questions the system cannot answer
Unbounded growth	An unbounded store stays cheap	Cost and (eventually) latency climb over the system's lifetime

Proposed response (each traces to a D1 capability and a D2 decision)

Failure	Design change	Expected benefit	Trade-off introduced
Pollution (0.20 precision)	Admission gate (C3) + multi-signal ranking (C6); D2 idea A2	Fewer, better memories; higher precision	Write-path latency; tuning the gate
Stale wins (0.33)	Recency in ranking (C6) + conflict resolution (C8)	Current preference wins; auditable supersession	Must detect contradictions correctly
Leak (0.92)	Tenant isolation enforced at the data layer (C10)	Zero cross-tenant returns by construction	None acceptable to trade; this is a gate
PII (1.0)	PII filter on write + deletion path (C11)	Secrets never enter the store	Filter false-positives may drop benign data
No abstention	Relevance threshold; return nothing below it	Honest "I don't know" instead of a guess	Threshold tuning; some recall lost
Growth	Decay + archival (C9)	Bounded, faster, cheaper store over time	Risk of decaying a still-useful memory

Note on F4: the exact-fact miss did not appear, because TF-IDF keyword matching is strong on exact strings — which is precisely why D2 adopted hybrid (vector + keyword) retrieval. The keyword arm tested here *is* the proposed fix for F4.

Falsification — what would show a proposed response is unnecessary

Each fix is now a testable claim against this same baseline:

Fix	Result that would make it unnecessary or wrong
Admission + ranking	If precision@5 with the gate does not beat 0.20 after tuning, the gate adds cost without benefit.
Recency + conflict	If stale-wins does not drop below 0.33, recency weighting is not the cause and the rule is wrong.
Isolation	If leak rate is not driven to 0.00, the isolation is not enforced where it must be.
Abstention	If thresholding raises abstention without hurting answerable recall, it is free; if it costs recall, the threshold is mis-set.

Bottom line. The baseline reproduced five of six predicted failures as hard numbers, and the two it did not reproduce (latency, overflow) taught me something real: at this scale the enemy is *pollution and leakage*, not speed or window size. Every improvement in the D4 design now has a measured number to beat on this exact workload — which is the comparison the handbook makes a non-negotiable gate.